1

Why Do We Test Software?

The true subject matter of the tester is not testing, but the design of test cases.

The purpose of this book is to teach software engineers how to test. This knowledge is useful whether you are a programmer who needs to unit test your own software, a full-time tester who works mostly from requirements at the user level, a manager in charge of testing or development, or any position in between. As the software industry moves into the second decade of the 21st century, software quality is increasingly becoming essential to all businesses and knowledge of software testing is becoming necessary for all software engineers.

Today, software defines behaviors that our civilization depends on in systems such as network routers, financial calculation engines, switching networks, the Web, power grids, transportation systems, and essential communications, command, and control services. Over the past two decades, the software industry has become much bigger, is more competitive, and has more users. Software is an essential component of exotic embedded applications such as airplanes, spaceships, and air traffic control systems, as well as mundane appliances such as watches, ovens, cars, DVD players, garage door openers, mobile phones, and remote controllers. Modern households have hundreds of processors, and new cars have over a thousand; all of them running software that optimistic consumers assume will never fail! Although many factors affect the engineering of reliable software, including, of course, careful design and sound process management, testing is the primary way industry evaluates software during development. The recent growth in agile processes puts increased pressure on testing; unit testing is emphasized heavily and test-driven development makes tests key to functional requirements. It is clear that industry is deep into a revolution in what testing means to the success

of software products.

Fortunately, a few basic software testing concepts can be used to design tests for a large variety of software applications. A goal of this book is to present these concepts in such a way that students and practicing engineers can easily apply them to any software testing situation.

This textbook differs from other software testing books in several respects. The most important difference is in how it views testing techniques. In his landmark book *Software Testing Techniques*, Beizer wrote that testing is simple—all a tester needs to do is “find a graph and cover it.” Thanks to Beizer’s insight, it became evident to us that the myriad of testing techniques present in the literature have much more in common than is obvious at first glance. Testing techniques are typically presented in the context of a particular software artifact (for example, a requirements document or code) or a particular phase of the lifecycle (for example, requirements analysis or implementation). Unfortunately, such a presentation obscures underlying similarities among techniques.

This book clarifies these similarities with two innovative, yet simplifying, approaches. First, we show how testing is more efficient and effective by using a classical engineering approach. Instead of designing and developing tests on concrete software artifacts like the source code or requirements, we show how to develop abstraction models, design tests at the abstract level, and then implement actualtests at the concrete level by satisfying the abstract designs. This is the exact process that traditional engineers use, except whereas they usually use calculus and algebra to describe the abstract models, software engineers usually use discrete mathematics. Second, we recognize that all test criteria can be defined with a very short list of abstract models: input domain characterizations, graphs, logical expressions, and syntactic descriptions. These are directly reflected in the four chapters of Part II of this book.

This book provides a balance of theory and practical application, thereby presenting testing as a collection of objective, quantitative activities that can be measured and repeated. The theory is based on the published literature, and presented without excessive formalism. Most importantly, the theoretical concepts are presented when needed to support the practical activities that test engineers follow. That is, this book is intended for all software developers.

1.1 WHEN SOFTWARE GOES BAD

As said, we consider the development of software to be engineering. And like any engineering discipline, the software industry has its shares of failures, some spectacular, some mundane, some costly, and sadly, some that have resulted in loss of life. Before learning about software disasters, it is important to understand the difference between faults, errors, and failures. We adopt the definitions of software fault, error, and failure from the dependability community.

Definition 1.1 Software Fault: A static defect in the software.

Definition 1.2 Software Error: An incorrect internal state that is the manifestation of some fault.

Definition 1.3 Software Failure: External, incorrect behavior with respect to the requirements or another description of the expected behavior.

Consider a medical doctor diagnosing a patient. The patient enters the doctor's office with a list of *failures* (that is, symptoms). The doctor then must discover the *fault*, or root cause of the symptoms. To aid in the diagnosis, a doctor may order tests that look for anomalous internal conditions, such as high blood pressure, an irregular heartbeat, high levels of blood glucose, or high cholesterol. In our terminology, these anomalous internal conditions correspond to *errors*.

While this analogy may help the student clarify his or her thinking about faults, errors, and failures, software testing and a doctor's diagnosis differ in one crucial way. Specifically, faults in software are *design mistakes*. They do not appear spontaneously, but exist as a result of a decision by a human. Medical problems (as well as faults in computer system hardware), on the other hand, are often a result of physical degradation. This distinction is important because it limits the extent to which any process can hope to control software faults. Specifically, since no foolproof way exists to catch arbitrary mistakes made by humans, we can never eliminate all faults from software. In colloquial terms, we can make software development foolproof, but we cannot, and should not attempt to, make it damn-foolproof.

For a more precise example of the definitions of fault, error, and failure,

we need to clarify the concept of the state. A *program state* is defined during execution of a program as the current value of all live variables and the current location, as given by the program counter. The *program counter* (PC) is the next statement in the program to be executed and can be described with a line number in the file (*PC = 5*) or the statement as a string (*PC = "if (x > y)"*). Most of the time, what we mean by a statement is obvious, but complex structures such as *for* loops have to be treated specially. The program line "*for (i=1; i < N; i++)*" actually has three statements that can result in separate states. The loop initialization ("*i=1*") is separate from the loop test ("*i < N*"), and the loop increment ("*i++*") occurs at the end of the loop body. As an illustrative example, consider the following Java method:

```
/**
 * Counts zeroes in an array
 *
 * @param x array to count zeroes in
 * @return number of occurrences of 0 in x
 * @throws NullPointerException if x is null
 */
public static int numZero (int[] x)
{
   int count = 0;
   for (int i = 1; i < x.length; i++)
   {
      if (x[i] == 0) count++;
   }
   return count;
}
```

Sidebar

Programming Language Independence

This book strives to be independent of language, and most of the concepts in the book are. At the same time, we want to illustrate these concepts with specific examples. We choose Java, and emphasize that most of these examples would be very similar in many other common

languages.

The fault in this method is that it starts looking for zeroes at index 1 instead of index 0, as is necessary for arrays in Java. For example, `numZero ([2, 7, 0])` correctly evaluates to 1, while `numZero ([0, 7, 2])` incorrectly evaluates to 0. In both tests the faulty statement is executed. Although both of these tests result in an error, only the second results in failure. To understand the error states, we need to identify the state for the method. The statefor `numZero()` consists of values for the variables `x`, `count`, `i`, and the program counter (`PC`). For the first example above, the state at the loop test on the very first iteration of the loop is (`x` = [2, 7, 0], `count` = 0, `i` = 1, `PC` = “ `i < x.length`”). Notice that this state is erroneous precisely because the value of `i` should be zero on the first iteration. However, since the value of `count` is coincidentally correct, the error state does not propagate to the output, and hence the software does not fail. In other words, a state is in error simply if it is not the expected state, even if all of the values in the state, considered in isolation, are acceptable. More generally, if the required sequence of states is s_0, s_1, s_2, ..., and the actual sequence of states is s_0, s_2, s_3, ..., then state s_2 is in error in the second sequence. The fault model described here is quite deep, and this discussion gives the broad view without going into unneeded details. The exercises at the end of the section explore some of the subtleties of the fault model.

In the second test for our example, the error state is (`x` = [0, 7, 2], `count` = 0, `i` = 1, `PC` = “ `i < x.length`”). In this case, the error propagates to the variable `count` and is present in the return value of the method. Hence a failure results.

The term *bug* is often used informally to refer to all three of fault, error, and failure. This book will usually use the specific term, and avoid using “bug.” A favorite story of software engineering teachers is that Grace Hopper found a moth stuck in a relay on an early computing machine, which started the use of bug as a problem with software. It is worth noting, however, that the term bug has an old and rich history, predating software by at least a century. The first use of bug to generally mean a problem we were able to find is from a quote by Thomas Edison :

It has been just so in all of my inventions. The first step is an intuition, and

comes with a burst, then difficulties arise–this thing gives out and [it is] then that 'Bugs'–as such little faults and difficulties are called–show themselves and months of intense watching, study and labor are requisite.
— Thomas Edison

A very public failure was the Mars lander of September 1999, which crashed due to a misunderstanding in the units of measure used by two modules created by separate software groups. One module computed thruster data in English units and forwarded the data to a module that expected data in metric units. This is a very typical integration fault (but in this case enormously expensive, both in terms of money and prestige).

One of the most famous cases of software killing people is the Therac-25 radiation therapy machine. Software faults were found to have caused at least three deaths due to excessive radiation. Another dramatic failure was the launch failure of the first Ariane 5 rocket, which exploded 37 seconds after liftoff in 1996. The low-level cause was an unhandled floating point conversion exception in an inertial guidance system function. It turned out that the guidance system could never encounter the unhandled exception when used on the Ariane 4 rocket. That is, theguidance system function was correct for Ariane 4. The developers of the Ariane 5 quite reasonably wanted to reuse the successful inertial guidance system from the Ariane 4, but no one reanalyzed the software in light of the substantially different flight trajectory of the Ariane 5. Furthermore, the system tests that would have found the problem were technically difficult to execute, and so were not performed. The result was spectacular–and expensive!

The famous Pentium bug was an early alarm of the need for better testing, especially unit testing. Intel introduced its Pentium microprocessor in 1994, and a few months later, Thomas Nicely, a mathematician at Lynchburg College in Virginia, found that the chip gave incorrect answers to certain floating-point division calculations.

The chip was slightly inaccurate for a few pairs of numbers; Intel claimed (probably correctly) that only one in nine billion division operations would exhibit reduced precision. The fault was the omission of five entries in a table of 1, 066 values (part of the chip's circuitry) used by a division algorithm. The five entries should have contained theconstant +2, but the entries were not initialized and contained zero instead. The MIT mathematician Edelman claimed that "the bug in the Pentium was an

easy mistake to make, and a difficult one to catch," an analysis that misses an essential point. This was a very difficult mistake to find during system testing, and indeed, Intel claimed to have run millions of tests using this table. But the table entries were left empty because a loop termination condition was incorrect; that is, the loop stopped storing numbers before it was finished. Thus, this would have been a very simple fault to find during unit testing; indeed analysis showed that almost any unit level coverage criterion would have found this multimillion dollar mistake.

The great northeast blackout of 2003 was started when a power line in Ohio brushed against overgrown trees and shut down. This is called a *fault* in the power industry. Unfortunately, the software alarm system failed in the local power company, so system operators could not understand what happened. Other lines also sagged into trees and switched off, eventually overloading other power lines, which then cut off. This cascade effect eventually caused a blackout throughout southeastern Canada and eight states in the northeastern part of the US. This is considered the biggest blackout in North American history, affecting 10 million people in Canada and 40 million in the USA, contributing to at least 11 deaths and costing up to $6 billion USD.

Some software failures are felt widely enough to cause severe embarrassment to the company. In 2011, a centralized students data management system in Korea miscalculated the academic grades of over 29, 000 middle and high school students. This led to massive confusion about college admissions and a government investigation into the software engineering practices of the software vendor, Samsung Electronics.

A 1999 study commissioned by the U.S. National Research Council and the U.S. President's commission on critical infrastructure protection concluded that the current base of science and technology is inadequate for building systems to control critical software infrastructure. A 2002 report commissioned by the National Institute of Standards and Technology (NIST) estimated that defective software costs the U.S. economy $59.5 billion per year. The report further estimated that 64% of the costs were a result of user mistakes and 36% a result of design and development mistakes, and suggested that improvements in testing could reduce this cost by about a third, or $22.5 billion. Blumenstyk reported that web application failures lead to huge losses in businesses; $150, 000 per hour in media companies, $2.4 million per hour in credit card sales, and $6.5 million per hour in the financial services market.

Software faults do not just lead to functional failures. According to a Symantec security threat report in 2007, 61 percent of all vulnerabilities disclosed were due to faulty software. The most common are web application vulnerabilities that can be attacked by some common attack techniques using invalid inputs.

These public and expensive software failures are getting more common and more widely known. This is simply a symptom of the change in expectations of software. As we move further into the 21st century, we are using more safety critical, real-time software. Embedded software has become ubiquitous; many of us carry millions of lines of embedded software in our pockets. Corporations rely more and more on large-scale enterprise applications, which by definition have large user bases and high reliability requirements. Security, which used to depend on cryptography, then database security, then avoiding network vulnerabilities, is now largely about avoiding software faults. The Web has had a major impact. It features a deployment platform that offers software services that are very competitive and available to millions of users. They are also distributed, adding complexity, and must be highly reliable to be competitive. More so than at any previous time, industry desperately needs to apply the accumulated knowledge of over 30 years of testing research.

1.2 GOALS OF TESTING SOFTWARE

Surprisingly, many software engineers are not clear about their testing goals. Is it to show correctness, find problems, or something else? To explore this concept, we first must separate validation and verification. Most of the definitions in this book are taken from standards documents, and although the phrasing is ours, we try to be consistent with the standards. Useful standards for reading in more detail are the IEEE Standard Glossary of Software Engineering Terminology, DOD-STD-2167A and MIL-STD-498 from the US Department of Defense, and the British Computer Society's Standard for Software Component Testing.

Definition 1.4 Verification: The process of determining whether the products of a phase of the software development process fulfill the requirements established during the previous phase.

Definition 1.5 Validation: The process of evaluating software at the

end of software development to ensure compliance with intended usage.

Verification is usually a more technical activity that uses knowledge about the individual software artifacts, requirements, and specifications. Validation usually depends on domain knowledge; that is, knowledge of the application for which the software is written. For example, validation of software for an airplane requires knowledge from aerospace engineers and pilots.

As a familiar example, consider a light switch in a conference room. Verification asks if the lighting meets the specifications. The specifications might say something like, “The lights in front of the projector screen can be controlled independently of the other lights in the room.” If the specifications are written down somewhere and thelights **cannot** be controlled independently, then the lighting fails verification, precisely because the implementation does not satisfy the specifications. Validation asks whether users are satisfied, an inherently fuzzy question that has nothing to do with verification. If the “independent control” specification is neither written down nor satisfied, then, despite the disappointed users, verification nonetheless succeeds, because the implementation satisfies the specification. But validation fails, because the specification for the lighting does not reflect the true needs of the users. This is an important general point: validation exposes flaws in specifications.

The acronym “IV&V” stands for “Independent Verification and Validation,” where “independent” means that the evaluation is done by non-developers. Sometimes the IV&V team is within the same project, sometimes the same company, and sometimes it is entirely an external entity. In part because of the independent nature of IV&V, the process often is not started until the software is complete and is often done by people whose expertise is in the application domain rather than software development. This can sometimes mean that validation is given more weight than verification. This book emphasizes verification more than validation, although most of the specific test criteria we discuss can be used for both activities.

Beizer discussed the goals of testing in terms of the “test process maturity levels” of an organization, where the levels are characterized by the testers’ goals. He defined five levels, where the lowest level is not worthy of being given a number.

Level 0	There is no difference between testing and debugging.
Level 1	The purpose of testing is to show correctness.
Level 2	The purpose of testing is to show that the software does not work.
Level 3	The purpose of testing is not to prove anything specific, but to reduce the risk of using the software.
Level 4	Testing is a mental discipline that helps all IT professionals develop higher- quality software.

Level 0 is the view that testing is the same as debugging. This is the view that is naturally adopted by many undergraduate Computer Science majors. In most CS programming classes, the students get their programs to compile, then debug the programs with a few inputs chosen either arbitrarily or provided by the professor. This model does not distinguish between a program's incorrect behavior and a mistake within the program, and does very little to help develop software that is reliable or safe.

In **Level 1** testing, the purpose is to show correctness. While a significant step up from the naive level 0, this has the unfortunate problem that in any but the most trivial of programs, correctness is virtually impossible to either achieve or demonstrate. Suppose we run a collection of tests and find no failures. What do we know? Should we assume that we have good software or just bad tests? Since the goal of correctness is impossible, test engineers usually have no strict goal, real stopping rule, or formal test technique. If a development manager asks how much testing remains to be done, the test manager has no way to answer the question. In fact, test managers are in a weak position because they have no way to quantitatively express or evaluate theirwork.

In **Level 2** testing, the purpose is to show failures. Although looking for failures is certainly a valid goal, it is also inherently negative. Testers may enjoy finding the problem, but the developers never want to find

problems–they want the software to work (yes, level 1 thinking can be natural for the developers). Thus, level 2 testing puts testers and developers into an adversarial relationship, which can be bad for team morale. Beyond that, when our primary goal is to look for failures, we are still left wondering what to do if no failures are found. Is our work done? Is our software very good, or is the testing weak? Having confidence in when testing is complete is an important goal for all testers. It is our view that this level currently dominates the software industry.

The thinking that leads to **Level 3** testing starts with the realization that testing can show the presence, but not the absence, of failures. This lets us accept the fact that whenever we use software, we incur some risk. The risk may be small and the consequences unimportant, or the risk may be great and the consequences catastrophic, but risk is always there. This allows us to realize that the entire development team wants the same thing–to reduce the risk of using the software. In level 3 testing, both testers and developers work together to reduce risk. We see more and more companies move to this testing maturity level every year.

Once the testers and developers are on the same "team," an organization can progress to real **Level 4** testing. Level 4 thinking defines testing as *a mental discipline that increases quality*. Various ways exist to increase quality, of which creating tests that cause the software to fail is only one. Adopting this mindset, test engineers can become the technical leaders of the project (as is common in many other engineering disciplines). They have the primary responsibility of measuring and improving software quality, and their expertise should help the developers. Beizer used the analogy of a spell checker. We often think that the purpose of a spell checker is to find misspelled words, but in fact, the best purpose of a spell checker is to improve our ability to spell. Every time the spell checker finds an incorrectly spelled word, we have the opportunity to learn how to spell the word correctly. The spell checker is the "expert" on spelling quality. In the same way, level 4 testing means that the purpose of testing is to improve the ability of the developers to produce high-quality software. The testers should be the experts who train your developers!

As a reader of this book, you probably start at level 0, 1, or 2. Most software developers go through these levels at some stage in their careers. If you work in software development, you might pause to reflect on which testing level describes your company or team. The remaining chapters in Part I should help you move to level 2 thinking, and to understand the

importance of level 3. Subsequent chapters will give you the knowledge, skills, and tools to be able to work at level 3. An ultimate goal of this book is to provide a philosophical basis that will allow readers to become "change agents" in their organizations for level 4 thinking, and test engineers to become **software quality experts**. Although level 4 thinking is currently rare in the software industry, it is common in more mature engineeringfields.

These considerations help us decide at a strategic level why we test. At a more tactical level, it is important to know why **each test** is present. If you do not know why you are conducting each test, the test will not be very helpful. What fact is each test trying to verify? It is essential to document test objectives and test requirements, including the planned coverage levels. When the test manager attends a planning meeting with the other managers and the project manager, the test manager must be able to articulate clearly how much testing is enough and when testing will complete. In the 1990s, we could use the "date criterion," that is, testing is "complete" when the ship date arrives or when the budget is spent.

Figure 1.1 dramatically illustrates the advantages of testing early rather than late. This chart is based on a detailed analysis of faults that were detected and fixed during several large government contracts. The bars marked'**A**' indicate what percentage of faults *appeared* in that phase. Thus, 10% of faults appeared during the requirements phase, 40% during design, and 50% during implementation. The bars marked '**D**' indicated the percentage of faults that were *detected* during each phase. About 5% were detected during the requirements phase, and over 35% during system testing. Lastly is the cost analysis. The solid bars marked '**C**' indicate the relative *cost* of finding and fixing faults during each phase. Since each project was different, this is averaged to be based on a "unit cost." Thus, faults detected and fixed during requirements, design, and unit testing were a single unit cost. Faults detected and fixed during integration testing cost five times as much, 10 times as much during system testing, and 50 times as much after the software is deployed.

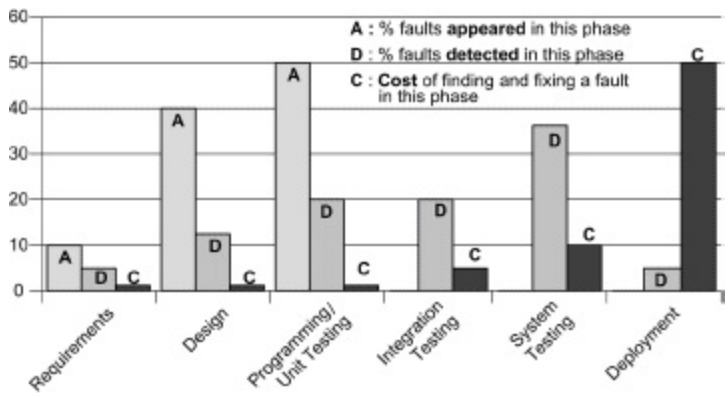

Figure 1.1. Cost of late testing.

If we take the simple assumption of $1000 USD unit cost per fault, and 100 faults, that means we spend $39, 000 to find and correct faults during requirements, design, and unit testing. During integration testing, the cost goes upto $100, 000. But system testing and deployment are the serious problems. We find more faults during system testing at ten times the cost, for a total of $360, 000. And even though we only find a few faults after deployment, the cost being 50 X unit means we spend $250, 000! Avoiding the work early (requirements analysis and unit testing) saves money in the short term. But it leaves faults in software that are like little bombs, ticking away, and the longer they tick, the bigger the explosion when they finally go off.

To put Beizer's level 4 test maturity level in simple terms, the goal of testing is to eliminate faults as **early** as possible. We can never be perfect, but every time we eliminate a fault during unit testing (or sooner!), we save money. The rest of this book will teach you how to do that.

EXERCISES

Chapter 1.

1. What are some factors that would help a development organization move from Beizer's testing level 2 (*testing is to show errors*) to testing level 4 (*a mental discipline that increases quality*)?
2. What is the difference between software **fault** and software **failure**?
3. What do we mean by "*level 3 thinking is that the purpose of testing is to reduce risk?*" What risk? Can we reduce the risk to zero?
4. The following exercise is intended to encourage you to think of testing in a more rigorous way than you may be used to. The exercise

also hints at the strong relationship between specification clarity, faults, and test cases[1].

(a) Write a Java method with the signature

```
public static Vector union (Vector a, Vector b)
```

The method should return a Vector of objects that are in either of the two argument Vectors.

(b) Upon reflection, you may discover a variety of defects and ambiguities in the given assignment. In other words, ample opportunities for faults exist. Describe as many possible faults as you can. (*Note:* `Vector` *is a Java* `Collection` *class. If you are using another language, interpret* `Vector` *as a list.*)

(c) Create a set of test cases that you think would have a reasonable chance of revealing the faults you identified above. Document a rationale for each test in your test set. If possible, characterize all of your rationales in some concise summary. Run your tests against your implementation.

(d) Rewrite the method signature to be precise enough to clarify the defects and ambiguities identified earlier. You might wish to illustrate your specification with examples drawn from your test cases.

5. Below are four faulty programs. Each includes test inputs that result in failure. Answer the following questions about each program.

```java
/**
 * Find last index of element
 *
 * @param x array to search
 * @param y value to look for
 * @return last index of y in x; -1 if absent
 * @throws NullPointerException if x is null
 */
public int findLast (int[] x, int y)
{
  for (int i=x.length-1; i > 0; i--)
  {
    if (x[i] == y)
    {
      return i;
    }
  }
  return -1;
}
// test:  x = [2, 3, 5]; y = 2; Expected = 0
// Book website: FindLast.java
// Book website: FindLastTest.java
```

```java
/**
 * Find last index of zero
 *
 * @param x array to search
 *
 * @return last index of 0 in x; -1 if absent
 * @throws NullPointerException if x is null
 */
public static int lastZero (int[] x)
{
  for (int i = 0; i < x.length; i++)
  {
    if (x[i] == 0)
    {
      return i;
    }
  }
  return -1;
}
// test:  x = [0, 1, 0]; Expected = 2
// Book website: LastZero.java
// Book website: LastZeroTest.java
```

```java
/**
 * Count positive elements
 *
 * @param x array to search
 * @return count of positive elements in x
 * @throws NullPointerException if x is null
 */
public int countPositive (int[] x)
{
  int count = 0;
  for (int i=0; i < x.length; i++)
  {
    if (x[i] >= 0)
    {
      count++;
    }
  }
  return count;
}
// test:  x = [-4, 2, 0, 2]; Expected = 2
// Book website: CountPositive.java
// Book website: CountPositiveTest.java
```

```java
/**
 * Count odd or positive elements
 *
 * @param x array to search
 * @return count of odd/positive values in x
 * @throws NullPointerException if x is null
 */
public static int odd0rPos(int[] x)
{
  int count = 0;
  for (int i = 0; i < x.length; i++)
  {
    if (x[i]%2 == 1 || x[i] > 0)
    {
      count++;
    }
  }
  return count;
}
// test:  x = [-3, -2, 0, 1, 4]; Expected = 3
// Book website: Odd0rPos.java
// Book website: Odd0rPosTest.java
```

(a) Explain what is wrong with the given code. Describe the fault precisely by proposing a modification to the code.

(b) If possible, give a test case that does **not** execute the fault. If not, briefly explain why not.

(c) If possible, give a test case that executes the fault, but does **not** result in an error state. If not, briefly explain why not.

(d) If possible give a test case that results in an error, but **not** a failure. If not, briefly explain why not. Hint: Don't forget about the program counter.

(e) For the given test case, describe the first error state. Be sure to describe the complete state.

(f) Implement your repair and verify that the given test now produces the expected output. Submit a screen printout or other evidence that your new program works.

6. Answer question (a) or (b), **but not both**, depending on your background.

   (a) If you do, or have, worked for a software development company, what level of test maturity do you think the company worked at? (0: testing=debugging, 1: testing shows correctness, 2: testing shows the program doesn't work, 3: testing reduces risk, 4: testing is a mental discipline about quality).

   (b) If you have **never** worked for a software development company, what level of test maturity do you think that **you** have? (0: testing=debugging, 1: testing shows correctness, 2: testing shows the program doesn't work, 3: testing reduces risk, 4: testing is a mental discipline about quality).

7. Consider the following three example classes. These are OO faults taken from Joshua Bloch's *Effective Java*, Second Edition. Answer the following questions about each.

```
class Vehicle implements Cloneable
{
  private int x;
  public Vehicle (int y) { x = y;}
  public Object clone()
  {
    Object result = new Vehicle (this.x);
    // Location "A"
    return result;
  }
  // other methods omitted
}
class Truck extends Vehicle
{
  private int y;
  public Truck (int z) { super (z); y = z;}
  public Object clone()
```

```
  {
    Object result = super.clone();
    // Location "B"
    ((Truck) result).y = this.y;  // throws ClassCastException
    return result;
  }
  // other methods omitted
}
//Test: Truck suv = new Truck (4); Truck co = suv.clone()
//      Expected: suv.x = co.x; suv.getClass() = co.getClass()
```

Note: Relevant to Bloch, Item 11 page 54.
Book website: Vehicle.java, Truck.java, CloneTest.java

```
public class BigDecimalTest
{
  BigDecimal x = new BigDecimal ("1.0");
  BigDecimal y = new BigDecimal ("1.00");
  // Fact:  !x.equals (y), but x.compareTo (y) == 0

  Set <BigDecimal> BigDecimalTree = new TreeSet <BigDecimal> ();
  BigDecimalTree.add (x);
  BigDecimalTree.add (y);
  // TreeSet uses compareTo(), so BigDecimalTree now has 1 element

  Set <BigDecimal> BigDecimalHash = new HashSet <BigDecimal> ();
  BigDecimalHash.add (x);
  BigDecimalHash.add (y);
  // HashSet uses equals(), so BigDecimalHash now has 2 elements
}
  // Test: System.out.println ("BigDecimalTree = " + BigDecimalTree);
        // System.out.println ("BigDecimalHash = " + BigDecimalHash);
        // Expected: BigDecimalTree = 1; BigDecimalHash = 1

// See Java Doc for add() in Set Interface
// The problem is that in BigDecimal, equals() and compareTo()
// are inconsistent. Let's suppose we decide that compareTo() is correct,
// and that equals()is faulty.
```

Note: Relevant to Bloch, Item 12 page 62.
Book website: class BigDecimalTest.java

```
class Point
{
   private int x; private int y;
   public Point (int x, int y) { this.x=x; this.y=y; }

   @Override public boolean equals (Object o)
   {
      // Location A
      if (!(o instanceof Point)) return false;
      Point p = (Point) o;
      return (p.x == this.x) && (p.y == this.y);
   }
}
class ColorPoint extends Point
{
   private Color color;
   // Fault: Superclass instantiable; subclass state extended

   public ColorPoint (int x, int y, Color color)
   {
      super (x,y);
      this.color = color;
   }
```

```
   @Override public boolean equals (Object o)
   {
      // Location B
      if (!(o instanceof ColorPoint)) return false;
      ColorPoint cp = (ColorPoint) o;
      return (super.equals(cp) && (cp.color == this.color));
   }
   // Tests:
      Point p  = new Point (1,2);
      ColorPoint cp1   = new ColorPoint (1,2,RED);
      ColorPoint cp2   = new ColorPoint (1,2,BLUE);
      p.equals (cp1);    // Test 1: Result = true;
      cp1.equals (p);    // Test 2: Result = false;
      cp1.equals (cp2);  // Test 3: Result = false;
   // Expected: p.equals (cp1) = true; cp1.equals (p) = true,
   //        cp1.equals (cp2) = false

Note: Relevant to Bloch, Item 17 page 87.
Book website: Point.java, ColorPoint.java, PointTest.java
```

(a) Explain what is wrong with the given code. Describe the fault precisely by proposing a modification to the code.

(b) If possible, give a test case that does **not** execute the fault. If not, briefly explain why not.

(c) If possible, give a test case that executes the fault, but does **not**

result in an error state. If not, briefly explain why not.

(d) If possible give a test case that results in an error, but **not** a failure. If not, briefly explain why not. Hint: Don't forget about the program counter.

(e) In the given code, describe the first error state. Be sure to describe the complete state.

(f) Implement your repair and verify that the given test now produces the expected output. Submit a screen printout or other evidence that your new program works.

1.3 BIBLIOGRAPHIC NOTES

This textbook has been deliberately left uncluttered with references. Instead, each chapter contains a Bibliographic Notes section, which contains suggestions for further and deeper reading for readers who want more. We especially hope that research students will find these sections helpful.

Most of the terminology in testing is from standards documents, including the IEEE Standard Glossary of Software Engineering Terminology [IEEE, 2008], the US Department of Defense [Department of Defense, 1988, Department of Defense, 1994], the US Federal Aviation Administration FAA-DO178B, and the British Computer Society's Standard for Software Component Testing [British Computer Society, 2001].

Beizer [Beizer, 1990] first defined the testing levels in Section 1.2. Beizer described them in terms of the maturity of individual developers and used the term *phase* instead of *level*. We adapted the discussion to organizations rather than individual developers and chose the term *level* to mirror the language of the well-known Capability Maturity Model [Paulk et al., 1995].

All books on software testing and all researchers owe major thanks to the landmark books in 1979 by Myers [Myers, 1979], in 1990 by Beizer [Beizer, 1990], and in 2000 by Binder [Binder, 2000]. Some excellent overviews of unit testing criteria have also been published, including one by White [White, 1987] and more recently by Zhu, Hall, and May [Zhu et al., 1997]. The recent text from Pezze and Young [Pezze and Young, 2008] reports relevant processes, principles, and techniques from the

testing literature, and includes many useful classroom materials. The Pezze and Young text presents coverage criteria in the traditional lifecycle-based manner, and does not organize criteria into the four abstract models discussed in this chapter. Another recent book by Mathur offers a comprehensive, in-depth catalog of test techniques and criteria [Mathur, 2014].

Numerous other software testing books were not intended as textbooks, or do not offer general coverage for classroom use. Beizer's *Software System Testing and Quality Assurance* [Beizer, 1984] and Hetzel's *The Complete Guide to Software Testing* [Hetzel, 1988] cover various aspects of management and process for software testing. Several books cover specific aspects of testing [Howden, 1987, Marick, 1995, Roper, 1994]. The STEP project at Georgia Institute of Technology resulted in a comprehensive survey of the practice of software testing by Department of Defense contractors in the 1980s [DeMillo et al., 1987].

The information for the Pentium bug and Mars lander was taken from several sources, including by Edelman, Moler, Nuseibeh, Knutson, and Peterson [Edelman, 1997, Knutson and Carmichael, 2000, Moler, 1995, Nuseibeh, 1997, Peterson, 1997]. The well-written official accident report [Lions, 1996] is our favorite source for understanding the details of the Ariane 5 Flight 501 Failure. The information for the Therac-25 accidents was taken from Leveson and Turner's deep analysis [Leveson and Turner, 1993]. The details on the 2003 Northeast Blackout was taken from Minkel's analysis in Scientific American [Minkel, 2008] and Rice's book [Rice, 2008]. The information about the Korean education information system was taken from two newspaper articles [Min-sang and Sang-soo, 2011, Korea Times, 2011].

The 1999 study mentioned was published in an NRC / PITAC report [PITAC, 1999, Schneider, 1999]. The data in Figure 1.1 were taken from a NIST report that was developed by the Research Triangle Institute [RTI, 2002]. The figures on web application failures are due to Blumenstyk [Blumenstyk, 2006]. The figures about faulty software leading to security vulnerabilities are from Symantec [Symantec, 2007].

Finally, Rick Hower's QATest website is a good resource for current, elementary, information about software testing: www.softwareqatest.com.

---

[1] Liskov's *Program Development in Java*, especially chapters 9 and 10, is a great

source for students who wish to learn more about this.

2

Model-Driven Test Design

Designers are more efficient and effective if they can raise their level of abstraction.

This chapter introduces one of the major innovations in the second edition of Introduction to Software Testing. Software testing is inherently complicated and our ultimate goal, completely correct software, is unreachable. The reasons are formal (as discussed below in section 2.1) and philosophical. As discussed in Chapter 1, it's not even clear that the term "correctness" means anything when applied to a piece of engineering as complicated as a large computer program. Do we expect correctness out of a building? A car? A transportation system? Intuitively, we know that all large physical engineering systems have problems, and moreover, there is no way to say what correct means. This is even more true for software, which can quickly get orders of magnitude more complicated than physical structures such as office buildings or airplanes.

Instead of looking for "correctness," wise software engineers try to evaluate software's "behavior" to decide if the behavior is acceptable within consideration of a large number of factors including (but not limited to) reliability, safety, maintainability, security, and efficiency. Obviously this is more complex than the naive desire to show the software is correct.

So what do software engineers do in the face of such overwhelming complexity? The same thing that physical engineers do–we use mathematics to "raise our level of abstraction. " The Model-Driven Test Design (MDTD) process breaks testing into a series of small tasks that simplify test generation. Then test designers isolate their task, and work at a higher level of abstraction by using mathematical engineering structures to design test values independently of the details of software or design artifacts, test automation, and test execution.

A key intellectual step in MDTD is test case design. Test case design can be the primary determining factor in whether tests successfully find failures in software. Tests can be designed with a "human-based" approach, where a test engineer uses domain knowledge of the software's purpose and his or her experience to design tests that will be effective at finding faults. Alternatively, tests can be designed to satisfy well-defined engineering goals such as coverage criteria. This chapter describes the task activities and then introduces criteria-based test design. Criteria-based test design will be discussed in more detail in Chapter 5, then specific criteria on four mathematical structures are described in Part II. After these preliminaries, the model-driven test design process is defined in detail. The book website has simple web applications that support the MDTD in the context of the mathematical structures in Part II.

2.1 SOFTWARE TESTING FOUNDATIONS

One of the most important facts that all software testers need to know is that testing can show only the presence of failures, not their absence. This is a fundamental, theoretical limitation; to be precise, the problem of finding all failures in a program is undecidable. Testers often call a test successful (or effective) if it finds an error. While this is an example of level 2 thinking, it is also a characterization that is often useful and that we will use throughout the book. This section explores some of the theoretical underpinnings of testing as a way to emphasize how important the MDTD is.

The definitions of fault and failure in Chapter 1 allow us to develop the reachability, infection, propagation, and revealability model ("RIPR"). First, we distinguish testing from debugging.

Definition 2.6 Testing: Evaluating software by observing its execution.

Definition 2.7 Test Failure: Execution of a test that results in a software failure.

Definition 2.8 Debugging: The process of finding a fault given a failure.

Of course the central issue is that for a given fault, not all inputs will "trigger" the fault into creating incorrect output (a failure). Also, it is often very difficult to relate a failure to the associated fault. Analyzing these ideas leads to the fault/failure model, which states that four conditions are needed for a failure to be observed.

Figure 2.1 illustrates the conditions. First, a test must reach the location or locations in the program that contain the fault (*Reachability*). After the location is executed, the state of the program must be incorrect (*Infection*). Third, the infected state must propagate through the rest of the execution and cause some output or final state of the program to be incorrect (*Propagation*). Finally, the tester must observe part of the incorrect portion of the final program state (*Revealability*). If the tester only observes parts of the correct portion of the final program state, the failure is not revealed. This is shown in the cross-hatched intersection in Figure 2.1. Issues with revealing failures will be discussed in Chapter 4 when we present test automation strategies.

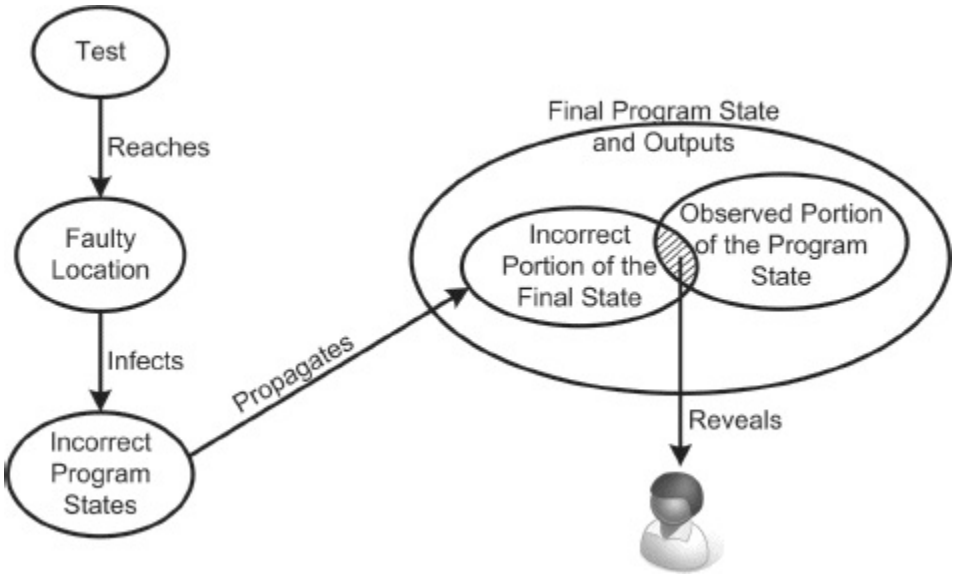

Figure 2.1. Reachability, Infection, Propagation, Revealability (RIPR) model.

Collectively, these four conditions are known as the fault/failure model, or the *RIPR* model.

It is important to note that the RIPR model applies even when the fault is missing code (so-called faults of omission). In particular, when execution passes through the location where the missing code should be, the program counter, which is part of the program state, necessarily has the wrong value.

From a practitioner's view, these limitations mean that software testing is complex and difficult. The common way to deal with complexity in engineering is to use abstraction by abstracting out complicating details that can be safely ignored by modeling the problem with some mathematical structures. That is a central theme of this book, which we begin by analyzing the separate technical activities involved in creating good tests.

2.2 SOFTWARE TESTING ACTIVITIES

In this book, a *test engineer* is an Information Technology (IT) professional who is in charge of one or more technical test activities, including designing test inputs, producing test case values, running test scripts, analyzing results, and reporting results to developers and managers. Although we cast the description in terms of test engineers, every engineer involved in software development should realize that he or she sometimes wears the hat of a test engineer. The reason is that each software artifact produced over the course of a product's development has, or should have, an associated set of test cases, and the person best positioned to define these test cases is often the designer of the artifact. A *test manager* is in charge of one or more test engineers. Test managers set test policies and processes, interact with other managers on the project, and otherwise help the engineers test software effectively and efficiently.

Figure 2.2 shows some of the major activities of test engineers. A test engineer must design tests by creating test requirements. These requirements are then transformed into actual values and scripts that are ready for execution. These executable tests are run against the software, denoted `P` in the figure, and the results are evaluated to determine if the tests reveal a fault in the software. These activities may be carried out by one person or by several, and the process is monitored by a test manager.

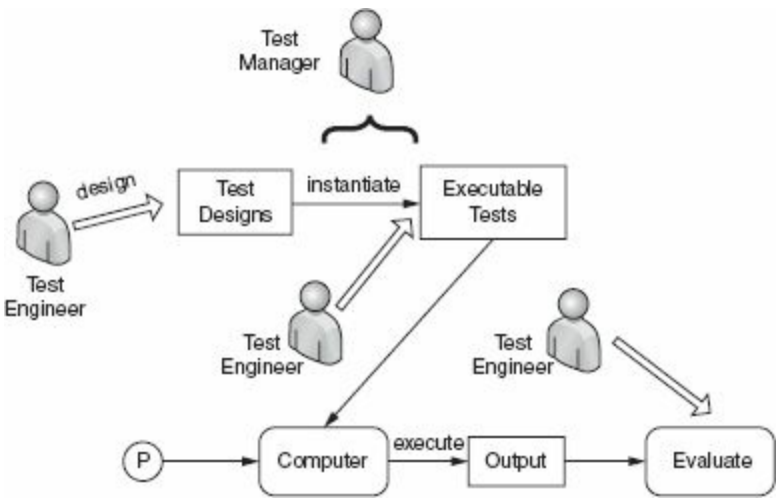

Figure 2.2. Activities of test engineers.

One of a test engineer's most powerful tools is a formal coverage criterion. Formal coverage criteria give test engineers ways to decide what test inputs to use during testing, making it more likely that the tester will find problems in the program and providing greater assurance that the software is of high quality and reliability. Coverage criteria also provide stopping rules for the test engineers. The technical core of this book presents the coverage criteria that are available, describes how they are supported by tools (commercial and otherwise), explains how they can best be applied, and suggests how they can be integrated into the overall development process.

Software testing activities have long been categorized into levels, and the most often used level categorization is based on traditional software process steps. Although most types of tests can only be run after some part of the software is implemented, tests can be designed and constructed during all software development steps. The most time-consuming parts of testing are actually the test design and construction, so test activities can and should be carried out throughout development.

2.3 TESTING LEVELS BASED ON SOFTWARE ACTIVITY

Tests can be derived from requirements and specifications, design artifacts, or the source code. In traditional texts, a different level of testing accompanies each distinct software development activity:

- Acceptance Testing : assess software with respect to requirements or users' needs.
- System Testing : assess software with respect to architectural design and overall behavior.
- Integration Testing : assess software with respect to subsystem design.
- Module Testing: assess software with respect to detailed design.
- Unit Testing : assess software with respect to implementation.

Figure 2.3, often called the "V model," illustrates a typical scenario for testing levels and how they relate to software development activities by isolating each step. Information for each test level is typically derived from the associated development activity. Indeed, standard advice is to design the tests concurrently with each development activity, even though the software will not be in an executable form until the implementation phase. The reason for this advice is that the mere process of designing tests can identify defects in design decisions that otherwise appear reasonable. Early identification of defects is by far the best way to reduce their ultimate cost. Note that this diagram is **not** intended to imply a waterfall process. The synthesis and analysis activities generically apply to any development process.

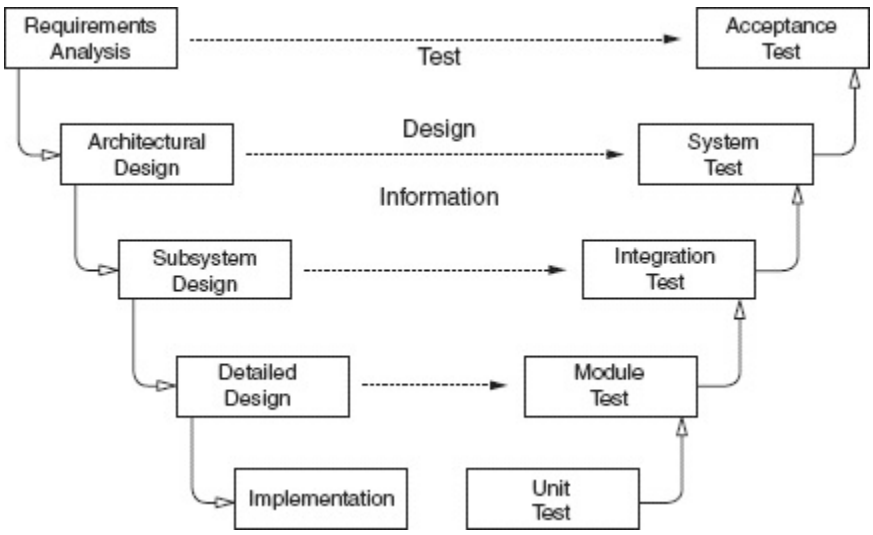

Figure 2.3. Software development activities and testing levels – the "V Model".

The *requirements analysis* phase of software development captures the customer's needs. *Acceptance testing* is designed to determine whether the completed software in fact meets these needs. In other words, acceptance testing probes whether the software does what the users want. Acceptance

testing must involve users or other individuals who have strong domain knowledge.

The *architectural design* phase of software development chooses components and connectors that together realize a system whose specification is intended to meet the previously identified requirements. *System testing* is designed to determine whether the assembled system meets its specifications. It assumes that the pieces work individually, and asks if the system works as a whole. This level of testing usually looks for design and specification problems. It is a very expensive place to find lower-level faults and is usually not done by the programmers, but by a separate testing team.

The *subsystem design* phase of software development specifies the structure and behavior of subsystems, each of which is intended to satisfy some function in the overall architecture. Often, the subsystems are adaptations of previously developed software. *Integration testing* is designed to assess whether the interfaces between modules (defined below) in a subsystem have consistent assumptions and communicate correctly. Integration testing must assume that modules work correctly. Some testing literature uses the terms integration testing and system testing interchangeably; in this book, integration testing does **not** refer to testing the integrated system or subsystem. Integration testing is usually the responsibility of members of the development team.

The *detailed design* phase of software development determines the structure and behavior of individual modules. A *module* is a collection of related units that are assembled in a file, package, or class. This corresponds to a file in C, a package in Ada, and a class in C++ and Java. *Module testing* is designed to assess individual modules in isolation, including how the component units interact with each other and their associated data structures. Most software development organizations make module testing the responsibility of the programmer; hence the common term *developer testing*.

Implementation is the phase of software development that actually produces code. A program *unit*, or procedure, is one or more contiguous program statements, with a name that other parts of the software use to call it. Units are called functions in C and C++, procedures or functions in Ada, methods in Java, and subroutines in Fortran. *Unit testing* is designed to assess the units produced by the implementation phase and is the “lowest” level of testing. In some cases, such as when building general-

purpose library modules, unit testing is done without knowledge of the encapsulating software application. As with module testing, most software development organizations make unit testing the responsibility of the programmer, again, often called developer testing. It is straightforward to package unit tests together with the corresponding code through the use of tools such as *JUnit* for Java classes.

Because of the many dependencies among methods in classes, it is common among developers using object-oriented (OO) software to combine unit and module testing and use the term unit testing or developertesting.

Not shown in Figure 2.3 is regression testing, a standard part of the maintenance phase of software development. *Regression testing* is done after changes are made to the software, to help ensure that the updated software still possesses the functionality it had before the updates.

Mistakes in requirements and high-level design end up being implemented as faults in the program; thus testing can reveal them. Unfortunately, the software faults that come from requirements and design mistakes are visible only through testing months or years after the original mistake. The effects of the mistake tend to be dispersed throughout multiple software components; hence such faults are usually difficult to pin down and expensive to correct. On the positive side, even if tests cannot be executed, the very process of defining tests can identify a significant fraction of the mistakes in requirements and design. Hence, it is important for test planning to proceed concurrently with requirements analysis and design and not be put off until late in a project. Fortunately, through techniques such as use case analysis, test planning is becoming better integrated with requirements analysis in standard software practice.

Although most of the literature emphasizes these levels in terms of **when** they are applied, a more important distinction is on the **types of faults** that we are looking for. The faults are based on the software **artifact** that we are testing, and the software **artifact** that we derive the tests from. For example, unit and module tests are derived to test units and modules, and we usually try to find faults that can be found when executing the units and modules individually.

One final note is that OO software changes the testing levels. OO software blurs the distinction between units and modules, so the OO software testing literature has developed a slight variation of these levels. *Intra-method testing* evaluates individual methods. *Inter-method testing*

evaluates pairs of methods within the same class. *Intra-class testing* evaluates a single entire class, usually as sequences of calls to methods within the class. Finally, *inter-class testing* evaluates more than one class at the same time. The first three are variations of unit and module testing, whereas inter-class testing is a type of integration testing.

2.4 COVERAGE CRITERIA

The essential problem with testing is the numbers. Even a small program has a huge number of possible inputs. Consider a tiny method that computes the average of three integers. We have only three input variables, but each can have any value between -MAXINT and +MAXINT. On a 32-bit machine, each variable has a possibility of over **4 billion** values. With three inputs, this means the method has over **80 Octillion** possible inputs!

So no matter whether we are doing unit testing, integration testing, or system testing, it is impossible to test with all inputs. The input space is, to all practical purposes, infinite. Thus a test designer's goal could be summarized in a very high-level way as searching a huge input space, hoping to find the fewest tests that will reveal the most problems. This is the source of two key problems in testing: (1) how do we search? and (2) when do we stop? Coverage criteria give us structured, practical ways to search the input space. Satisfying a coverage criterion gives a tester some amount of confidence in two crucial goals: (A) we have looked in many corners of the input space, and (B) our tests have a fairly low amount of overlap.

Coverage criteria have many advantages for improving the quality and reducing the cost of test data generation. Coverage criteria can maximize the "bang for the buck," with fewer tests that are effective at finding more faults. Well-designed criteria-based tests will be comprehensive, yet factor out unwanted redundancy. Coverage criteria also provide traceability from software artifacts such as source, design models, requirements, and input space descriptions. This supports regression testing by making it easier to decide which tests need to be reused, modified, or deleted. From an engineering perspective, one of the strongest benefits of coverage criteria is they provide a "stopping rule" for testing; that is, we know in advance approximately how many tests are needed and we know when we have

“enough” tests. This is a powerful tool for engineers and managers.

Coverage criteria also lend themselves well to automation. As we will formalize in Chapter 5, a test requirement is a specific element of a software artifact that a test case must satisfy or cover, and a coverage criterion is a rule or collection of rules that yield test requirements. For example, the coverage criterion “cover every statement” yields one test requirement for each statement. The coverage criterion “cover every functional requirement” yields one test requirement for each functional requirement. Test requirements can be stated in semi-formal, mathematical terms, and then manipulated algorithmically. This allows much of the test data design and generation process to be automated.

The research literature presents a lot of overlapping and identical coverage criteria. Researchers have invented hundreds of criteria on dozens of software artifacts. However, if we **abstract** these artifacts into mathematical models, many criteria turn out to be exactly the same. For example, the idea of covering pairs of edges in finite state machines was first published in 1976, using the term *switch cover*. Later, the same idea was applied to control flow graphs and called *two-trip*, still again, the same idea was “invented” for state transition diagrams and called *transition-pair* (we define this formally using the generic term edge-pair in Chapter 7). Although they looked very different in the research literature, if we generalize these structures to graphs, all three ideas are the same. Similarly, node coverage and edge coverage have each been defined dozens of times.

Sidebar

Black-Box and White-Box Testing

Black-box testing and the complementary white-box testing are old and widely used terms in software testing. In black-box testing, we derive tests from external descriptions of the software, including specifications, requirements, and design. In white-box testing, on the other hand, we derive tests from the source code internals of the software, specifically including branches, individual conditions, and statements. This somewhat arbitrary distinction started to lose coherence when the term gray-box testing was applied to developing tests from design elements, and the approach taken in this book eliminates the need for the distinction altogether.

Some older sources say that white-box testing is used for system testing and black-box testing for unit testing. This distinction is certainly false, since all testing techniques considered to be white-box can be used at the system level, and all testing techniques considered to be black-box can be used on individual units. In reality, unit testers are currently more likely to use white-box testing than system testers are, simply because white-box testing requires knowledge of the program and is more expensive to apply, costs that can balloon on a large system.
This book relies on developing tests from mathematical abstractions such as graphs and logical expressions. As will become clear in Part II, these structures can be extracted from any software artifact, including source, design, specifications, or requirements. Thus asking whether a coverage criterion is black-box or white-box is the wrong question. One more properly should ask from what level of abstraction is the structure drawn.

In fact, all test coverage criteria can be boiled down to a few dozen criteria on just **four** mathematical structures: input domains, graphs, logic expressions, and syntax descriptions (grammars). Just like mechanical, civil, and electrical engineers use calculus and algebra to create abstract representations of physical structures, then solve various problems at this abstract level, software engineers can use discrete math to create abstract representations of software, then solve problems such as test design.

The core of this book is organized around these four structures, as reflected in the four chapters in Part II. This structure greatly simplifies teaching test design, and our classroom experience with the first edition of this book helped us realize this structure also leads to a simplified testing process. This process allows test design to be abstracted and carried out efficiently, and also separates test activities that need different knowledge and skill sets. Because the approach is based on these four abstract models, we call it the Model-Driven Test Design process (MDTD).

Sidebar

MDTD and Model-Based Testing

Model-based testing (MBT) is the design of software tests from an abstract model that represents one or more aspects of the software. The model usually, but not always, represents some aspects of the behavior

of the software, and sometimes, but not always, is able to generate expected outputs. The models are often described with UML diagrams, although more formal models as well as other informal modeling languages are also used. MBT typically assumes that the model has been built to specify the behavior of the software and was created during a design stage of development.

The ideas presented in this book are not, strictly speaking, exclusive to model-based testing. However, there is much overlap with MDTD and most of the concepts in this book can be directly used as part of MBT.

Specifically, we derive our tests from abstract structures that are very similar to models. An important difference is that these structures can be created ***after*** *the software is implemented, by the tester as part of test design. Thus, the structures do not specify behavior; they represent behavior. If a model was created to specify the software behavior, a tester can certainly use it, but if not, a tester can create one. Second, we create idealized structures that are more abstract than most modeling languages. For example, instead of UML statecharts or Petri nets, we design our tests from graphs. If model-based testing is being used, the graphs can be derived from a graphical model. Third, model-based testing explicitly does not use the source code implementation to design tests. In this book, abstract structures can be created from the implementation via things like control flow graphs, call graphs, and conditionals in decision statements.*

2.5 MODEL-DRIVEN TEST DESIGN

Academic teachers and researchers have long focused on the design of tests. We define *test design* to be the process of creating input values that will effectively test software. This is the most mathematical and technically challenging part of testing, however, academics can easily forget that this is only a small part of testing.

The job of developing tests can be divided into four discrete tasks: test design, test automation, test execution, and test evaluation. Many organizations assign the same person to all tasks. However, each task requires different skills, background knowledge, education, and training. Assigning the same person to all these tasks is akin to assigning the same software developer to requirements, design, implementation, integration,

and configuration control. Although this was common in previous decades, few companies today assign the same engineers to all development tasks. Engineers specialize, sometimes temporarily, sometimes for a project, and sometimes for their entire career. But should test organizations still assign the same people to all test tasks? They require different skills, and it is unreasonable to expect all testers to be good at all tasks, so this clearly wastes resources. The following subsections analyze each of these tasks in detail.

2.5.1 Test Design

As said above, test design is the process of designing input values that will effectively test software. In practice, engineers use two general approaches to designing tests. In *criteria-based test design*, we design test values that satisfy engineering goals such as coverage criteria. In *human-based test design*, we design test values based on domain knowledge of the program and human knowledge of testing. These are quite different activities.

Criteria-based test design is the most technical and mathematical job in software testing. To apply criteria effectively, the tester needs knowledge of discrete math, programming, and testing. That is, this requires much of a traditional degree in computer science. For somebody with a degree in computer science or software engineering, this is intellectually stimulating, rewarding, and challenging. Much of the work involves creating abstract models and manipulating them to design high-quality tests. In software development, this is analogous to the job of software architect; in building construction, this is analogous to the job of construction engineer. If an organization uses people who are not qualified (that is, do not have the required knowledge), they will spend time creating ineffective tests and be dissatisfied at work.

Human-based test design is quite different. The testers must have knowledge of the software's application domain, of testing, and of user interfaces. Human-based test designers explicitly attempt to find *stress tests*, tests that stress the software by including very large or very small values, boundary values, invalid values, or other values that the software may not expect during typical behavior. Human-based testers also explicitly consider actions the users might do, including unusual actions. This is much harder than developers may think and more necessary than

many test researchers and educators realize. Although criteria-based approaches often implicitly include techniques such as stress testing, they can be blind to special situations, and may miss problems that human-based tests would not. Although almost no traditional CS is needed, an empirical background (biology or psychology) or a background in logic (law, philosophy, math) is helpful. If the software is embedded on an airplane, a human-based test designer should understand piloting; if the software runs an online store, the test designers should understand marketing and the products being sold. For people with these abilities, human-based test design is intellectually stimulating, rewarding, and challenging–but often **not** to typical CS majors, who usually want to build software!

Many people think of criteria-based test design as being used for unit testing and human-based test design as being used for system testing. However, this is an artificial distinction. When using criteria, a graph is just a graph and it does not matter if it started as a control flow graph, a call graph, or an activity diagram. Likewise, human-based tests can and should be used to test individual methods and classes. The main point is that the approaches are complementary and we need both to fully test software.

2.5.2 Test Automation

The final result of test design is input values for the software. *Test automation* is the process of embedding test values into executable scripts. Note that automated tool support for test design is **not** considered to be test automation. This is necessary for efficient and frequent execution of tests. The programming difficulty varies greatly by the software under test (SUT). Some tests can be automated with basic programming skills, whereas if the software has low controllability or observability (for example, with embedded, real-time, or web software), test automation will require more knowledge and problem-solving skills. The test automator will need to add additional software to access the hardware, simulate conditions, or otherwise control the environment. However, many domain experts using human-based testing do not have programming skills. And many criteria-based test design experts find test automation boring. If a test manager asks a domain expert to automate tests, the expert is likely to

resist and do poorly; if a test manager asks a criteria-based test designer to automate tests, the designer is quite likely to go looking for a development job.

2.5.3 Test Execution

Test execution is the process of running tests on the software and recording the results. This requires basic computer skills and can often be assigned to interns or employees with little technical background. If all tests are automated, this is trivial. However, few organizations have managed to achieve 100% test automation. If tests must be run by hand, this becomes the most time-consuming testing task. Hand-executed tests require the tester to be meticulous with bookkeeping. Asking a good test designer to hand execute tests not only wastes a valuable (and possibly highly paid) resource, the test designer will view it as a very tedious job and will soon look for other work.

2.5.4 Test Evaluation

Test evaluation is the process of evaluating the results of testing and reporting to developers. This is **much** harder than it may seem, especially reporting the results to developers. Evaluating the results of tests requires knowledge of the domain, testing, user interfaces, and psychology. The knowledge required is very much the same as for human-based test designers. If tests are well-automated, then most test evaluation can (and should) be embedded in the test scripts. However, when automation is incomplete or when correct output cannot neatly be encoded in assertions, this task gets more complicated. Typical CS or software engineering majors will not enjoy this job, but to the right person, this is intellectually stimulating, rewarding, and challenging.

2.5.5 Test Personnel and Abstraction

These four tasks focus on designing, implementing and running the tests. Of course, they do not cover all aspects of testing. This categorization

omits important tasks like test management, maintenance, and documentation, among others. We focus on these because they are essential to developing test values.

A challenge to using criteria-based test design is the amount and type of knowledge needed. Many organizations have a shortage of highly technical test engineers. Few universities teach test criteria to undergraduates and many graduate classes focus on theory, supporting research rather than practical application. However, the good news is that with a well-planned division of labor, a single criteria-based test designer can support a fairly large number of test automators, executors and evaluators.

The model-driven test design process explicitly supports this division of labor. This process is illustrated in Figure 2.4, which shows test design activities above the line and other test activities below.

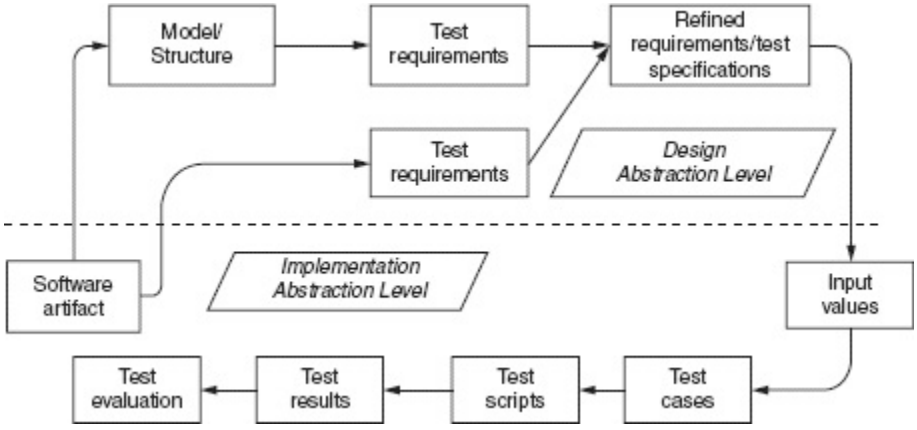

Figure 2.4. Model-driven test design.

The MDTD lets test designers “raise their level of abstraction ” so that a small subset of testers can do the mathematical aspects of designing and developing tests. This is analogous to construction design, where one engineer creates a design that is followed by many carpenters, plumbers, and electricians. The traditional testers and programmers can then do their parts: finding values, automating the tests, running tests, and evaluating them. This supports the truism that “testers ain’t mathematicians.”

The starting point in Figure 2.4 is a software artifact. This could be program source, a UML diagram, natural language requirements, or even a user manual. A criteria-based test designer uses that artifact to create an abstract model of the software in the form of an input domain, a graph, logic expressions, or a syntax description. Then a coverage criterion is

applied to create test requirements. A human-based test designer uses the artifact to consider likely problems in the software, then creates requirements to test for those problems. These requirements are sometimes refined into a more specific form, called the *test specification*. For example, if edge coverage is being used, a test requirement specifies which edge in a graph must be covered. A refined test specification would be a complete path through the graph.

Once the test requirements are refined, input values that satisfy the requirements must be defined. This brings the process down from the *design abstraction level* to the *implementation abstraction level*. These are analogous to the abstract and concrete tests in the model-based testing literature. The input values are augmented with other values needed to run the tests (including values to reach the point in the software being tested, to display output, and to terminate the program). The test cases are then automated into test scripts (when feasible and practical), run on the software to produce results, and results are evaluated. It is important that results from automation and execution be used to feed back into test design, resulting in additional or modified tests.

This process has two major benefits. First, it provides a clean separation of tasks between test design, automation, execution and evaluation. Second, raising our abstraction level makes test design **much** easier. Instead of designing tests for a messy implementation or complicated design model, we design at an elegant mathematical level of abstraction. This is exactly how algebra and calculus has been used in traditional engineering for decades.

Figure 2.5 illustrates this process for unit testing of a small Java method. The Java source is shown on the left, and its control flow graph is in the middle. This is a standard control flow graph with the initial node marked as a dotted circle and the final nodes marked as double circles (this notation will be defined rigorously in Chapter 7). The nodes are annotated with the source statements from the method for convenience.

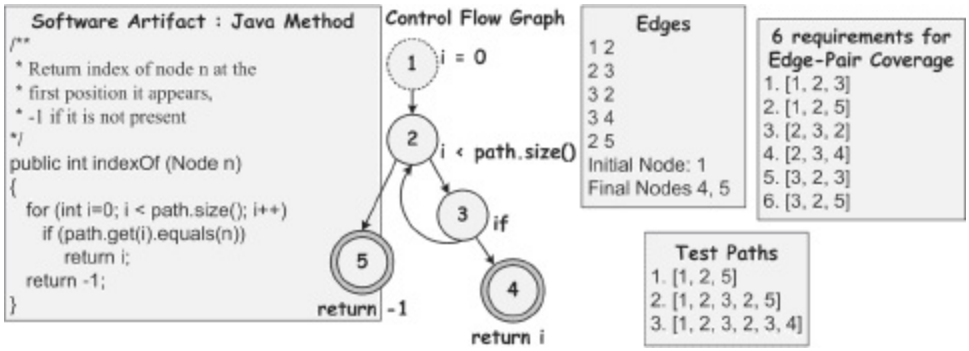

Figure 2.5. Example method, CFG, test requirements and test paths.

The first step in the MDTD process is to take this software artifact, the *indexOf()* method, and model it as an abstract structure. The control flow graph from Figure 2.5 is turned into an abstract version. This graph can be represented textually as a list of edges, initial nodes, and final nodes, as shown in Figure 2.5 under **Edges**. If the tester uses edge-pair coverage, (fully defined in Chapter 7), six requirements are derived. For example, test requirement #3, **[2, 3, 2]**, means the subpath from node 2 to 3 and back to 2 must be executed. The **Test Paths** box shows three complete test paths through the graph that will cover all six test requirements.

2.6 WHY MDTD MATTERS

The MDTD represents several years of introspection and deep thinking about the meaning and role of software testing. The first key insight was that the definitions and applications of test criteria are independent of the level of testing (unit, integration, system, etc.). This led to a powerful abstraction process that greatly simplifies testing, and was a major innovation of the first edition of this book. The analogy to the role of algebra and calculus in traditional engineering gives very strong support to the long-term viability of this idea.

This insight led us to a broader understanding of software testing activities and tasks. The separation of human-based and criteria-based test design is an important distinction, and the recognition that they are complementary, not competitive, activities is key to this book. All too often, academic researchers focus on criteria-based test design without respecting human-based test design, and practitioners and consultants focus on human-based test design without regard to criteria-based test design. Unfortunately this artificial split has reduced communication to the

detriment of the field.

Figure 2.4 illustrates how viewing test design as separate from test construction and execution can help distinguish test activities in meaningful ways, and combine them in an efficient process. Just as with software development and most traditional engineering activities, different people can be assigned to different activities. This allows test engineers to be more efficient, more effective, and have greater job satisfaction.

The four structures mentioned in Section 2.4 form the heart of this book. Each is used in a separate chapter in Part II to develop methods to design tests and to define criteria on the structures. The ordering in Part II follows the RIPR model of Section 2.1. The first structure, the input domain, is based on simple sets. The criteria in Chapter 6 help testers explore the input domain and do not explicitly satisfy any of the RIPR conditions. Chapter 7 uses graphs to design tests. The criteria require tests to "get to" certain places in the graph, thus satisfying *reachability*. Chapter 8 uses logic expressions to design tests. The criteria require tests to explore various truth assignments to the logic expressions, thus requiring that the tests not only reach the logic expressions, but also that they *infect* the state of the program. Chapter 9 uses grammars to design tests. These tests are not only required to reach locations and infect the program state, but the infection must also *propagate* to external behavior. Thus each chapter in Part II goes deeper into the RIPR model.

EXERCISES

Chapter 2.

1. How are faults and failures related to testing and debugging?
2. Answer question (a) or (b), **but not both**, depending on your background.
   (a) If you do, or have, worked for a software development company, how much effort did your testing / QA team put into each of the four test activities? (test design, automation, execution, evaluation)
   (b) If you have **never** worked for a software development company, which of the four test activities do you think you are best qualified for? (test design, automation, execution, evaluation)

2.7 BIBLIOGRAPHIC NOTES

The elementary result that finding all failures in a program is undecidable is due to Howden [Howden, 1976].

The fault/failure model was developed independently by Offutt and Morell in their dissertations [DeMillo and Offutt, 1993, Morell, 1990, Morell, 1984, Offutt, 1988]. Morell used the terms execution, infection, and propagation [Morell, 1984, Morell, 1990], and Offutt used reachability, sufficiency, and necessity [DeMillo and Offutt, 1993, Offutt, 1988]. This book merges the two sets of terms by using what we consider to be the most descriptive terms: reachability, infection, and propagation (RIP). The first edition of this book stopped there, but in 2014 Li and Offutt [Li and Offutt, 2016] extended the model by noting that automated test oracles necessarily only look at part of the output state. Even when the outputs are checked by hand, most humans will not be able to look at everything. Thus, the failure is only revealed to the tester if the tester looks at the "right" part of the output. Thus, this edition extends the old RIP model to the RIPR model.

Although this book does not focus heavily on the theoretical underpinnings of software testing, students interested in research should study such topics more in depth. A number of the papers are quite old, and often do not appear in current literature, and their ideas are beginning to disappear. The authors strongly encourage the study of the older papers. Among those are truly seminal papers in the 1970s by Goodenough and Gerhart [Goodenough and Gerhart, 1975] and Howden [Howden, 1976], and DeMillo, Lipton, Sayward, and Perlis [DeMillo et al., 1979, DeMillo et al., 1978]. These papers were followed up and refined by Weyuker and Ostrand [Weyuker and Ostrand, 1980], Hamlet [Hamlet, 1981], Budd and Angluin [Budd and Angluin, 1982], Gourlay [Gourlay, 1983], Prather [Prather, 1983], Howden [Howden, 1985], and Cherniavsky and Smith [Cherniavsky and Smith, 1986]. Later theoretical papers were contributed by Morell [Morell, 1984], Zhu [Zhu, 1996], and Wah [Wah, 1995, Wah, 2000]. Every PhD student's adviser will certainly have his or her own favorite theoretical papers.

The definition of *unit* is from Stevens, Myers and Constantine [Stevens et al., 1974], and the definition of *module* is from Sommerville [Sommerville, 1992]. The definition of *integration testing* is from Beizer [Beizer, 1990]. The clarification for OO testing levels with the terms *intra-*

method, *inter-method*, and *intra-class* testing is from Harrold and Rothermel [Harrold and Rothermel, 1994] and *inter-class* testing is from Gallagher, Offutt and Cincotta [Gallagher et al., 2007].

Pimont and Rault's *switch cover* paper was published in 1976 [Pimont and Rault, 1976]. The British Computer Society standard that used the term *two-trip* appeared in 1997 [British Computer Society, 2001]. Offutt et al.'s *transition-pair* paper was published in 2003 [Offutt et al., 2003].

The research literature on model-based testing is immense and growing, including a three-part special issue in Software Testing, Verification, and Reliability, edited by Ammann, Fraser, and Wotawa [Ammann et al., 2012a, Ammann et al., 2012b, Ammann et al., 2012c]. Rather than try to discuss all aspects of MBT, we suggest starting with Utting and Legeard's 2006 book, Practical Model-Based Testing [Utting and Legeard, 2006].

Good sources for issues about controllability and observability are Freedman [Freedman, 1991] and Binder [Binder, 2000].